



**UNIVERSIDAD AUTÓNOMA
DE CHIHUAHUA
FACULTAD DE INGENIERIA**

**COMPILADORES E INTERPRETES
8CC2**

3.15 Actividad 3: Introducción a Ray

**Isaac Gonzalez Aguilera
Luis Arturo Hernández Castillo
Angel Alonso López Quiñonez
Eduardo Tristán Chavez**

app.py

```
# pyrefly: ignore [missing-import]
import ray
import numpy as np
import requests
import os
import io
import zipfile
import subprocess

if not os.path.exists("accidentes/"):
    print("Descargando datos...")

url = "https://www.inegi.org.mx/contenidos/programas/accidentes/datosabiertos/
conjunto_de_datos_atu_anual_csv.zip"
response = requests.get(url)

with zipfile.ZipFile(io.BytesIO(response.content)) as z:
    z.extractall("accidentes/")
    print("Listo!")
else:
    print("Los datos ya existen, omitiendo descarga.")

print("Ejecutando procesar_datos.py...")
subprocess.run(["python", "procesar_datos.py"], check=True)

print("Iniciando Streamlit dashboard...")
subprocess.run(["streamlit", "run", "dashboard.py"], check=True)
```

- Aquí es donde comienza el proyecto, puesto que aquí primero que nada se revisa que haya datos descargados, y si no los llegara a haber entonces se llama a la api del INEGI y se descargan, así mismo, una vez que corrobora que hay datos es que se llama al primer subprocesso que es el de limpia de datos y procesamiento de datos, y una vez procesados los datos es que se llama al otro subprocesso para mostrar gráficamente los datos procesados.

procesar_datos.py

```
import os
import ray
import pyarrow as pa
import pandas as pd
import time

def process_csv_to_parquet():
    print("Inicializando Ray...")
    ray.init(address="ray://localhost:10001", ignore_reinit_error=True)
    input_dir_local = "accidentes/conjunto_de_datos"
    output_dir_local = "accidentes_parquet"
    if not os.path.exists(output_dir_local):
        os.makedirs(output_dir_local)

    print(f"Leyendo CSVs desde {input_dir_local}...")
    start_time = time.time()
    # Custom read with pandas in a Ray task to handle messy CSVs
    @ray.remote
    def process_file(filename):
        # Si corre dentro de Docker, usa la ruta del volumen (/app), si no, la local
        base_input = "/app/accidentes/conjunto_de_datos" if os.path.exists("/app/accidentes")
        else "accidentes/conjunto_de_datos"
        base_output = "/app/accidentes_parquet" if os.path.exists("/app/accidentes_parquet")
        else "accidentes_parquet"
        filepath = os.path.join(base_input, filename)
        try:
            # Read CSV. It has weird whitespaces and possibly a BOM in older files
            df = pd.read_csv(filepath, encoding="utf-8-sig", low_memory=False)
            # Clean column names
            df.columns = df.columns.str.strip()
            # Ensure columns are uppercase and clean
            df.columns = [col.replace("\uffff", "").upper() for col in df.columns]
            # Clean string columns: strip whitespaces
            for col in df.select_dtypes(['object']).columns:
                df[col] = df[col].astype(str).str.strip()
            # Filtrar filas que contengan "Se ignora" o "No especificado" en cualquier columna
            for col in df.columns:
                df = df[~df[col].astype(str).str.strip().str.lower().isin(['se ignora', 'no especificado'])]
            # Filtrar horas y meses inválidos ('99' suele representar valores ignorados)
            if 'ID_HORA' in df.columns:
                df = df[df['ID_HORA'].astype(str) != '99']
```

```

if 'MES' in df.columns:
df = df[df['MES'].astype(str) != '99']
# Convert numeric columns to numeric types safely
numeric_cols = ['CONDMUERTO', 'CONDHERIDO', 'PASAMUERTO', 'PASAHERIDO',
'PEATMUERTO', 'PEATHERIDO', 'CICLMUERTO', 'CICLHERIDO',
'OTROMUERTO', 'OTROHERIDO', 'NEMUERTO', 'NEHERIDO']
for col in numeric_cols:
if col in df.columns:
df[col] = pd.to_numeric(df[col], errors='coerce').fillna(0).astype(int)

# Output path
out_file = os.path.join(base_output, filename.replace('.csv', '.parquet'))
df.to_parquet(out_file, index=False)
return f"Procesado: {filename}"
except Exception as e:
return f"Error procesando {filename}: {str(e)}"

files = [f for f in os.listdir(input_dir_local) if f.endswith('.csv')]
futures = [process_file.remote(f) for f in files]
results = ray.get(futures)
for r in results:
print(r)
end_time = time.time()
print(f"Conversión a Parquet finalizada en {end_time - start_time:.2f} segundos.")
ray.shutdown()

if __name__ == "__main__":
process_csv_to_parquet()

```

- Esta parte comienza conectándose al clúster de Ray que levantaron los contenedores de Docker, luego verifica que exista la carpeta donde va a guardar los resultados y si no la crea. Después define la lógica de procesamiento que se va a distribuir entre los workers, dentro de la cual lo primero que hace es detectar automáticamente si está corriendo dentro de Docker o en local, y con base en eso ajusta las rutas de entrada y salida. Luego lee cada archivo CSV con pandas usando una codificación especial para manejar correctamente caracteres invisibles que algunos archivos del INEGI traen al inicio. Una vez leído, limpia los nombres de las columnas quitando espacios, caracteres raros y convirtiéndolos todos a mayúsculas para tener consistencia. Después recorre todas las columnas de texto y les quita espacios sobrantes, y filtra cualquier fila que contenga valores como "se ignora" o "no especificado" porque esos registros no sirven para el análisis. También descarta registros donde la hora o el mes tengan el código que usa el INEGI para indicar un dato desconocido. Finalmente convierte las columnas de víctimas a números enteros de forma segura, rellenando con cero lo que no se pueda convertir, y guarda el resultado

como un archivo Parquet que es mucho más eficiente que CSV para consultas analíticas. La parte distribuida ocurre cuando el script lista todos los CSVs disponibles y en lugar de procesarlos uno por uno, los encola todos en Ray al mismo tiempo para que los workers los procesen en paralelo, y al final espera a que todas las tareas terminen, imprime los resultados y apaga el clúster.

Dashboard.py

```
import streamlit as st
import pandas as pd
import os
import plotly.express as px

st.set_page_config(page_title="Dashboard de Accidentes Viales (INEGI)", layout="wide")

@st.cache_data
def load_data():
    data_dir = "accidentes_parquet"
    if not os.path.exists(data_dir):
        return pd.DataFrame()
    files = [f for f in os.listdir(data_dir) if f.endswith('.parquet')]
    dfs = []
    for f in files:
        try:
            # Leer cada parquet file
            df = pd.read_parquet(os.path.join(data_dir, f))
            # Mantener sólo las columnas relevantes para optimizar memoria
            cols_to_keep = [
                'ID_ENTIDAD', 'ID_MUNICIPIO', 'MES', 'ID_HORA', 'CAUSAACCI',
                'CONDMUERTO', 'CONDHHERIDO', 'PASAMUERTO', 'PASAHERIDO',
                'PEATMUERTO', 'PEATHERIDO', 'CICLMUERTO', 'CICLHERIDO',
                'OTROMUERTO', 'OTROHERIDO', 'NEMUERTO', 'NEHERIDO'
            ]
            cols = [c for c in cols_to_keep if c in df.columns]
            df = df[cols]
            dfs.append(df)
        except Exception as e:
            st.warning(f"Error al leer {f}: {e}")
    if dfs:
```

```

# Concatenar todos los DataFrames
full_df = pd.concat(dfs, ignore_index=True)
# Calcular totales de muertos y heridos
death_cols = ['CONDMUERTO', 'PASAMUERTO', 'PEATMUERTO', 'CICLMUERTO', 'OTROMUERTO',
'NEMUERTO']
injured_cols = ['CONDHERIDO', 'PASAHERIDO', 'PEATHERIDO', 'CICLHERIDO', 'OTROHERIDO',
'NEHERIDO']
# Asegurar que sean numéricos
for col in death_cols + injured_cols:
if col in full_df.columns:
full_df[col] = pd.to_numeric(full_df[col], errors='coerce').fillna(0)
else:
full_df[col] = 0
full_df['TOTAL_MUERTOS'] = full_df[death_cols].sum(axis=1)
full_df['TOTAL_HERIDOS'] = full_df[injured_cols].sum(axis=1)
return full_df
return pd.DataFrame()

```

```

# Cargar datos
with st.spinner('Cargando y procesando datos de accidentes (esto puede tomar unos segundos)...'):
df = load_data()

```

```

st.title("Sistema Distribuido para Análisis de Siniestros Viales")
st.markdown("Basado en datos abiertos de INEGI (ATUS). Esta plataforma responde a las preguntas planteadas en el proyecto.")

```

```

if df.empty:
st.error("No se encontraron datos. Por favor, asegúrate de correr `procesar_datos.py` primero.")
st.stop()

```

```

# Diccionarios para IDs de Estados
ESTADOS = {
1: 'Aguascalientes', 2: 'Baja California', 3: 'Baja California Sur',
4: 'Campeche', 5: 'Coahuila', 6: 'Colima',
7: 'Chiapas', 8: 'Chihuahua', 9: 'Ciudad de México',
10: 'Durango', 11: 'Guanajuato', 12: 'Guerrero',
13: 'Hidalgo', 14: 'Jalisco', 15: 'México',
16: 'Michoacán', 17: 'Morelos', 18: 'Nayarit',
19: 'Nuevo León', 20: 'Oaxaca', 21: 'Puebla',
22: 'Querétaro', 23: 'Quintana Roo', 24: 'San Luis Potosí',
25: 'Sinaloa', 26: 'Sonora', 27: 'Tabasco',
28: 'Tamaulipas', 29: 'Tlaxcala', 30: 'Veracruz',
31: 'Yucatán', 32: 'Zacatecas'
}

```

```
}
```

```
# Mapear estado
df['ESTADO'] = pd.to_numeric(df['ID_ENTIDAD'],
errors='coerce').map(ESTADOS).fillna("Desconocido")
df = df[df['ESTADO'] != "Desconocido"]
```

```
# Layout de Pestañas (Tabs)
tab1, tab2, tab3, tab4, tab5, tab6 = st.tabs([
"1. Estados", "2. Municipios", "3. Horarios",
"4. Causas", "5. Meses", "6. Gravedad"
])
```

```
with tab1:
st.header("1. ¿Qué estados concentran más accidentes?")
st.markdown("***Ranking por entidad federativa***")
acc_por_estado = df['ESTADO'].value_counts().reset_index()
acc_por_estado.columns = ['Estado', 'Total Accidentes']
fig1 = px.bar(acc_por_estado, x='Estado', y='Total Accidentes',
title='Accidentes por Estado', color='Total Accidentes',
color_continuous_scale='Reds')
st.plotly_chart(fig1, use_container_width=True)
with st.expander("Ver Datos Tabulares"):
st.dataframe(acc_por_estado, use_container_width=True)
```

```
with tab2:
st.header("2. ¿Qué municipios presentan mayor siniestralidad?")
st.markdown("***Tabla comparativa por municipio (Se muestra ID de municipio)***")
acc_por_mun = df.groupby(['ESTADO', 'ID_MUNICIPIO']).size().reset_index(name='Total
Accidentes')
acc_por_mun = acc_por_mun.sort_values(by='Total Accidentes', ascending=False).head(50)
st.write("Top 50 Municipios con más accidentes (Histórico)")
st.dataframe(acc_por_mun, use_container_width=True)
```

```
with tab3:
st.header("3. ¿En qué horarios ocurren más accidentes?")
st.markdown("***Distribución por hora del día***")
df['HORA'] = pd.to_numeric(df['ID_HORA'], errors='coerce')
df_horas = df[(df['HORA'] >= 0) & (df['HORA'] <= 23)]
acc_por_hora = df_horas['HORA'].value_counts().reset_index().sort_values('HORA')
acc_por_hora.columns = ['Hora', 'Total Accidentes']
fig3 = px.line(acc_por_hora, x='Hora', y='Total Accidentes', markers=True,
title='Accidentes por Hora del Día')
fig3.update_xaxes(dtick=1)
st.plotly_chart(fig3, use_container_width=True)
```

```

with tab4:
st.header("4. ¿Qué causas son más frecuentes?")
st.markdown("**Ranking de causas**")
# Algunas causas pueden estar vacías o ser NA
df['CAUSAACCI'] = df['CAUSAACCI'].fillna('No especificado')
acc_por_causa = df['CAUSAACCI'].value_counts().reset_index()
acc_por_causa.columns = ['Causa', 'Total Accidentes']
fig4 = px.pie(acc_por_causa, names='Causa', values='Total Accidentes',
title='Distribución de Causas de Accidentes', hole=0.3)
st.plotly_chart(fig4, use_container_width=True)

with tab5:
st.header("5. ¿Qué meses presentan mayor incidencia?")
st.markdown("**Tendencia mensual**")
df['MES_NUM'] = pd.to_numeric(df['MES'], errors='coerce')
df_meses = df[(df['MES_NUM'] >= 1) & (df['MES_NUM'] <= 12)]
meses_nombres = {1:'Enero', 2:'Febrero', 3:'Marzo', 4:'Abril', 5:'Mayo', 6:'Junio',
7:'Julio', 8:'Agosto', 9:'Septiembre', 10:'Octubre', 11:'Noviembre', 12:'Diciembre'}
acc_por_mes = df_meses['MES_NUM'].value_counts().reset_index().sort_values('MES_NUM')
acc_por_mes.columns = ['MesNum', 'Total Accidentes']
acc_por_mes['Mes'] = acc_por_mes['MesNum'].map(meses_nombres)
fig5 = px.bar(acc_por_mes, x='Mes', y='Total Accidentes', title='Accidentes por Mes', text='Total
Accidentes')
st.plotly_chart(fig5, use_container_width=True)

with tab6:
st.header("6. ¿Dónde hay más víctimas heridas o fallecidas?")
st.markdown("**Análisis por zona y gravedad**")
gravedad_estado = df.groupby('ESTADO')[['TOTAL_MUERTOS',
'TOTAL_HERIDOS']].sum().reset_index()
gravedad_estado['TOTAL_VICTIMAS'] = gravedad_estado['TOTAL_MUERTOS'] +
gravedad_estado['TOTAL_HERIDOS']
gravedad_estado = gravedad_estado.sort_values('TOTAL_VICTIMAS', ascending=False)
fig6 = px.bar(gravedad_estado.head(20), x='ESTADO', y=['TOTAL_HERIDOS', 'TOTAL_MUERTOS'],
title='Top 20 Estados con más Víctimas (Heridos y Fallecidos)',
barmode='stack', labels={'value': 'Total de Personas', 'variable': 'Tipo de Víctima'},
color_discrete_map={'TOTAL_HERIDOS': 'orange', 'TOTAL_MUERTOS': 'red'})
st.plotly_chart(fig6, use_container_width=True)
with st.expander("Ver Datos Tabulares"):
st.dataframe(gravedad_estado, use_container_width=True)

```

- Este programa carga todos los archivos Parquet generados por el procesamiento anterior, los une en un solo conjunto de datos y calcula el total de

muertos y heridos por accidente sumando las columnas de cada tipo de víctima. Con esos datos listos, presenta una interfaz de seis pestañas donde cada una responde una pregunta distinta: cuáles estados tienen más accidentes, qué municipios son los más peligrosos, a qué horas ocurren más siniestros, cuáles son las causas más frecuentes, en qué meses hay mayor incidencia, y finalmente dónde se concentran más víctimas entre heridos y fallecidos. Cada pestaña muestra una gráfica interactiva hecha con Plotly adecuada para ese tipo de análisis, ya sea de barras, líneas o pastel.